

CSS GIỚI THIỆU



★ CSS LÀ GÌ?

CSS (Cascading Style Sheets) là ngôn ngữ style sheet dùng để mô tả cách trình bày của một tài liệu được viết bằng **HTML**.

Nó kiểm soát **màu sắc, font chữ, layout, khoảng cách, background** và nhiều thứ khác nữa.

TẠI SAO DÙNG CSS?

- ✓ Tách nội dung (HTML) khỏi thiết kế
- ✓ Giúp website đẹp & đồng nhất
- ✓ Tiết kiệm thời gian và code (tái sử dụng)
- ✓ Dễ bảo trì
- ✓ Mạnh mẽ và linh hoạt

★ CSS HOẠT ĐỘNG NHƯ THẾ NÀO?



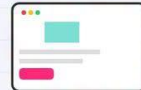
HTML
(Cấu trúc)

+



CSS
(Trình bày)

→



Trang web đẹp
(Kết quả)

✚ GHI NHỚ

HTML nói cho trình duyệt biết **CÁI GÌ** cần hiển thị.
CSS nói cho trình duyệt biết **HIỂN THỊ THẾ NÀO**. ❤️

CÚ PHÁP CSS

```
h1 {  
  color: blue;  
  font-size: 24px;  
}
```

Selector → chọn (các) phần tử HTML.

Declaration → một khối gồm một hoặc nhiều thuộc tính CSS.

Property → style bạn muốn thay đổi (vd: color, font-size).

Value → giá trị của property (vd: blue, 24px).

★ CÁC CÁCH THÊM CSS

1 INLINE CSS

Dùng thuộc tính style ngay trong thẻ HTML.

```
<p style="color: red;">Hello!</p>
```

2 INTERNAL (EMBEDDED) CSS

Viết CSS trong thẻ <style> ở phần <head>.

```
<style>  
  p { color: red; }  
</style>
```

3 EXTERNAL CSS

Viết CSS trong một file .css riêng và link tới HTML.

```
<link rel="stylesheet" href="style.css">
```

★ VÍ DỤ EXTERNAL CSS

index.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <link rel="stylesheet"  
    href="style.css">  
</head>  
<body>  
  <h1>Hello, CSS!</h1>  
  <p>This is a styled  
  paragraph.</p>  
</body>  
</html>
```

style.css

```
h1 {  
  color: purple;  
  font-size: 36px;  
}  
  
p {  
  color: #333;  
  font-size: 18px;  
  line-height: 1.6;  
}
```

KẾT QUẢ →

Hello, CSS!

This is a styled paragraph.

💡 MẸO NHANH

- ✓ Luôn giữ HTML cho cấu trúc và CSS cho styling.
- ✓ Dùng External CSS cho các dự án lớn.
- ✓ Đặt tên class có ý nghĩa.
- ✓ Giữ CSS gọn gàng, nhất quán. ❤️

★ ĐIỂM CHÍNH

- ★ CSS thêm style cho HTML.
- ★ Nó kiểm soát giao diện phần tử.
- ★ Giúp tiết kiệm thời gian, code sạch hơn.
- ★ Có 3 cách chính để thêm CSS.

TIẾP THEO

Cùng tìm hiểu về **CSS Selectors** ở Trang 2! 🚀

CSS SELECTORS

Selectors dùng để "chọn" các phần tử HTML dựa trên tên, id, class, thuộc tính và mối quan hệ để style chúng. ❤️



SELECTOR	CÚ PHÁP	VÍ DỤ HTML	VÍ DỤ CSS	KẾT QUẢ
1 ELEMENT SELECTOR Chọn tất cả phần tử có cùng tên thẻ.	<code>p { color: blue; }</code>	<code><p>This is a paragraph. </p> <p>This is another paragraph.</p></code>	<code>p { color: blue; }</code>	Tất cả thẻ <code><p></code> sẽ có màu xanh dương. This is a paragraph. This is another paragraph.
2 CLASS SELECTOR Chọn tất cả phần tử có thuộc tính class cho trước.	<code>.class { color: red; }</code>	<code><p class="note">Note 1</p> <p class="note">Note 2</p> Note 3</code>	<code>.note { color: red; }</code>	Tất cả phần tử có <code>class="note"</code> sẽ có màu đỏ. Note 1 Note 2 Note 3
3 ID SELECTOR Chọn phần tử có thuộc tính id cho trước.	<code>#id { color: green; }</code>	<code><h1 id="title">Hello CSS! </h1> <p id="title">This will not be green.</p></code>	<code>#title { color: green; }</code>	Chỉ phần tử có id="title" sẽ có màu xanh lá. Hello CSS!
4 UNIVERSAL SELECTOR Chọn tất cả phần tử trên trang.	<code>* { margin: 0; }</code>	<code><h1>Title</h1> <p>Paragraph</p> <div>Box</div></code>	<code>* { margin: 0; }</code>	Tất cả phần tử sẽ có margin: 0. h1 p div
5 GROUPING SELECTOR Chọn nhiều phần tử cách nhau bởi dấu phẩy.	<code>h1, p { color: purple; }</code>	<code><h1>Heading</h1> <p>Paragraph</p> Text</code>	<code>h1, p { color: purple; }</code>	Cả <code><h1></code> và <code><p></code> sẽ có màu tím. Heading Paragraph
6 DESCENDANT SELECTOR Chọn phần tử là hậu duệ của một phần tử khác.	<code>div p { color: teal; }</code>	<code><div> <p>Inside div</p> <p>Inside span</p> </div> <p>Outside div</p></code>	<code>div p { color: teal; }</code>	Tất cả <code><p></code> bên trong <code><div></code> sẽ có màu teal. Inside div Inside span Outside div
7 CHILD SELECTOR Chọn các con trực tiếp của một phần tử.	<code>ul > li { color: orange; }</code>	<code> Item 1 Item 2 Nested Item </code>	<code>ul > li { color: orange; }</code>	Chỉ các <code></code> con trực tiếp sẽ có màu cam. • Item 1 • Item 2 Nested Item
8 ADJACENT SIBLING SELECTOR Chọn phần tử ngay sau một phần tử cụ thể khác.	<code>h1 + p { color: brown; }</code>	<code><h1>Title</h1> <p>First paragraph</p> <p>Second paragraph</p></code>	<code>h1 + p { color: brown; }</code>	Chỉ <code><p></code> đầu tiên sau <code><h1></code> sẽ có màu nâu. First paragraph Second paragraph
9 GENERAL SIBLING SELECTOR Chọn tất cả anh em của một phần tử.	<code>h1 ~ p { color: darkblue; }</code>	<code><h1>Title</h1> <p>First paragraph</p> <div>Box</div> <p>Second paragraph</p></code>	<code>h1 ~ p { color: darkblue; }</code>	Tất cả <code><p></code> là anh em sau <code><h1></code> sẽ có màu xanh đậm. First paragraph Second paragraph
10 ATTRIBUTE SELECTOR Chọn phần tử có thuộc tính cụ thể.	<code>input[type="text"] { border: 1px solid #0077cc; }</code>	<code><input type="text" placeholder="Name"> <input type="password" placeholder="Pass"></code>	<code>input[type="text"] { border: 1px solid #0077cc; }</code>	Chỉ ô nhập text có viền xanh dương. [Name] [Pass]

MẸO NHANH

- ✓ ID selector là duy nhất. Chỉ dùng một lần trên trang.
- ✓ Class selector có thể tái sử dụng. Dùng để gom nhóm.
- ✓ Dùng tên class và id có ý nghĩa.
- ✓ Kết hợp selector để nhắm mục tiêu cụ thể hơn. ❤️

ĐỘ ĐẶC HIỆU CỦA SELECTOR (Cao → Thấp)

ID (#id) > CLASS, ATTRIBUTE, PSEUDO-CLASS (.class, [type], :hover) > ELEMENT, PSEUDO-ELEMENT (element, ::before, ::after) > UNIVERSAL (*)

GHI NHỚ

Selector càng cụ thể sẽ ghi đè selector kém cụ thể hơn nếu cùng nhắm vào một phần tử.

CSS MÀU SẮC, ĐƠN VỊ & GIÁ TRỊ

★ Màu sắc mang lại sức sống cho website của chúng ta. Đơn vị và giá trị giúp ta kiểm soát kích thước, khoảng cách và layout một cách chính xác. ♥

1 MÀU SẮC TRONG CSS

NAMED COLORS

140+ tên định sẵn.



red

blue

green

orange

purple

RGB COLORS

Red, Green, Blue (0-255).

rgb(255, 0, 0)

rgb(30, 144, 255)

rgb(46, 204, 113)

HEX COLORS

Dùng giá trị thập lục phân.



#FF0000

#1E90FF

#2ECC71

RGBA COLORS

RGB với Alpha (độ trong suốt 0-1).

rgba(255, 0, 0, 0.7)

rgba(30, 144, 255, 0.5)

rgba(46, 204, 113, 0.3)

HSL COLORS

Hue, Saturation, Lightness.

hsl(0, 100%, 50%)

hsl(210, 100%, 50%)

hsl(145, 60%, 45%)

HSLA COLORS

HSL với Alpha (độ trong suốt 0-1).

hsla(0, 100%, 50%, 0.7)

hsla(210, 100%, 50%, 0.5)

hsla(145, 60%, 45%, 0.3)

💡 Mẹo: Dùng công cụ online như coolors.co hoặc colorhunt.co để chọn bảng màu đẹp!

2 CÁC HÀM MÀU PHỔ BIẾN

color: rgb(255, 99, 71); ~ Màu đục
color: rgba(255, 99, 71, 0.5); ~ Trong suốt 50%
color: hsl(200, 100%, 50%); ~ Màu HSL
color: hsla(200, 100%, 50%, 0.4); ~ HSL có độ trong suốt

VÍ DỤ GRADIENT

linear-gradient(to right, #ff7e5f, #feb47b)

radial-gradient(circle, #6a1b9a, #2575fc)

3 ĐƠN VỊ TRONG CSS

px	Pixel cố định.
%	Phần trăm theo phân tử cha.
em	Tương đối theo font-size của cha.
rem	Tương đối theo font-size gốc (html).
vw	1% chiều rộng viewport.
vh	1% chiều cao viewport.
vmin	1% cạnh nhỏ hơn (width/height).
vmax	1% cạnh lớn hơn (width/height).
ch	Độ rộng ký tự "O".
ex	Chiều cao chữ "x" thường.

💡 GHI CHÚ NHANH

- ✓ Chọn màu hợp với thiết kế và thương hiệu.
- ✓ Dùng đơn vị tương đối (% , em, rem, vw, vh) cho website responsive.
- ✓ Dùng calc(), min(), max(), clamp() để tạo layout thông minh. ♥

HÀM CALC()

Thực hiện phép tính.

width: calc(100% - 40px);

MIN(), MAX(), CLAMP()

width: min(600px, 100%);
↳ Lấy giá trị nhỏ hơn.
width: max(300px, 50%);
↳ Lấy giá trị lớn hơn.
width: clamp(200px, 50%, 800px);
↳ Nằm giữa min và max.

★ BEST PRACTICES

- ★ Dùng HEX hoặc HSL để kiểm soát tốt hơn.
- ★ Dùng RGBA/HSLA cho độ trong suốt.
- ★ Ưu tiên rem hơn px cho font-size.
- ★ Dùng đơn vị tương đối cho responsive design.

🔥 GHI NHỚ

- Colors = tính thẩm mỹ
- Units = kiểm soát & khả năng responsive
- Values = kết quả chính xác hoặc linh hoạt 💡

CSS TEXT & FONTS

★ Các thuộc tính text trong CSS giúp kiểm soát hình thức, khoảng cách và căn chỉnh của text trên web. ♥

1 FONT FAMILY

Chỉ định kiểu chữ (typeface) cho text.

```
p {
  font-family: Arial, "Helvetica Neue",
  Helvetica, sans-serif;
}
```

ví dụ

Arial
Helvetica Neue
Georgia
"Times New Roman"
Courier New
Verdana

⚡ Luôn kết thúc bằng một họ chữ chung: serif | sans-serif | monospace | cursive | fantasy

2 FONT SIZE

Định nghĩa kích thước của text.

```
p {
  font-size: 18px;
}
```

Đơn vị phổ biến
px - Pixel cố định em - Tương đối cha
rem - Tương đối gốc % - Phần trăm
vw/vh - Tương đối viewport

ví dụ

10px: Text nhỏ
14px: Text thường
18px: Text lớn
24px: Text to hơn
32px: Text khổng lồ

3 FONT WEIGHT

Định độ đậm/mảnh của text.

```
p {
  font-weight: bold;
}
```

font-weight: 100 Mạnh
font-weight: 400 Bình thường
font-weight: 700 **Đậm**
font-weight: 900 **Rất đậm**

Giá trị phổ biến: normal (400) | bold (700) | bolder | lighter | 100-900

4 FONT STYLE

Định kiểu dáng của text.

```
p {
  font-style: italic;
}
```

normal - Con cáo nâu nhanh nhẹn
italic - Con cáo nâu nhanh nhẹn
oblique - Con cáo nâu nhanh nhẹn

Giá trị: normal | italic | oblique

5 LINE HEIGHT

Đặt chiều cao dòng của một khối text.

```
p {
  line-height: 1.6;
}
```

line-height: 1 (Chật)
Con cáo nâu nhảy qua con chó lười.

line-height: 1.6 (Vừa vặn)
Con cáo nâu nhảy qua con chó lười.

line-height: 2 (Rộng)
Con cáo nâu nhảy qua con chó lười.

⚡ Nên dùng giá trị không đơn vị.

6 TEXT ALIGN

Kiểm soát căn chỉnh ngang của text.

```
p {
  text-align: center;
}
```

left Con cáo nâu nhanh nhẹn.

center Con cáo nâu nhanh nhẹn.

right Con cáo nâu nhanh nhẹn.

justify Con cáo nâu nhảy qua con chó lười biếng nằm đó.

Giá trị: left | center | right | justify

7 TEXT DECORATION

Thêm trang trí cho text.

```
p {
  text-decoration:
  underline;
}
```

none: Không trang trí
underline: Text gạch chân
overline: Text gạch trên
line-through: ~~Text gạch giữa~~

8 TEXT TRANSFORM

Thay đổi kiểu chữ hoa/thường.

```
p {
  text-transform:
  uppercase;
}
```

uppercase: HELLO WORLD
lowercase: hello world
capitalize: Hello World
none: Hello World

9 LETTER-SPACING

Chỉnh khoảng cách giữa các ký tự.

```
p {
  letter-spacing: 2px;
}
```

0px: Text bình thường
2px: T e x t g i a n
5px: T e x t g i a n r o n g

10 WORD-SPACING

Chỉnh khoảng cách giữa các từ.

```
p {
  word-spacing: 8px;
}
```

0px: khoảng cách thường
8px: khoảng cách rộng hơn
16px: khoảng cách rộng hơn nhiều

11 WEB FONTS (GOOGLE FONTS)

Dùng font tùy chỉnh cho website.

- Vào fonts.google.com
- Chọn font và copy đoạn <link>
- Thêm vào <head> trong HTML
- Dùng trong CSS của bạn

```
<link href="https://fonts.googleapis.com/css2?
family=Poppins:wght@400;600&display=swap" rel="stylesheet">
```

```
body {
  font-family: 'Poppins', sans-serif;
}
```

MEO NHANH

- ✓ Dùng tối đa 2-3 font family.
- ✓ Dùng đơn vị tương đối (em, rem, %) để dễ scale.
- ✓ Đảm bảo độ tương phản tốt giữa text và nền.
- ✓ Line-height từ 1.4-1.8 để dễ đọc.
- ✓ Test font trên nhiều thiết bị. ♥

GHI NHỚ

Typography tốt giúp nội dung dễ đọc và đẹp mắt!

CSS BOX MODEL

★ Mọi phần tử HTML đều là một hình chữ nhật. CSS box model mô tả cấu trúc của hình chữ nhật này. ♥

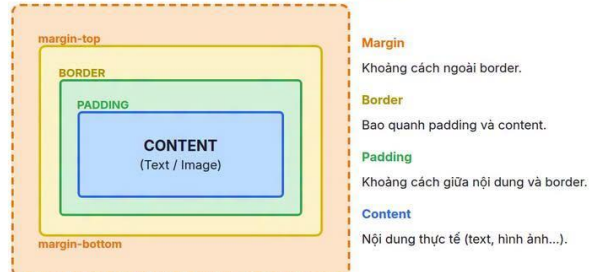
1 BOX MODEL LÀ GÌ?

Box model gồm:

- ✓ **Content** - Nội dung thực tế.
- ✓ **Padding** - Khoảng cách quanh nội dung.
- ✓ **Border** - Đường viền quanh padding.
- ✓ **Margin** - Khoảng cách ngoài border.

💡 Ghi nhớ: Margin nằm ngoài border và trong suốt (không có nền).

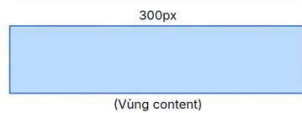
CẤU TRÚC BOX MODEL



2 WIDTH & HEIGHT

Mặc định, width và height chỉ áp dụng cho vùng content.

```
div {
  width: 300px;
  height: 150px;
}
```



3 BOX-SIZING

Kiểm soát cách tính width & height.

1) **content-box (mặc định)**
Width/height = chỉ vùng content.

```
div {
  box-sizing: content-box;
}
```

2) **border-box**
Width/height = content + padding + border.

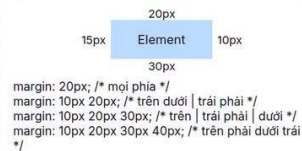
```
div {
  box-sizing: border-box;
}
```

💡 Dùng border-box giúp layout dễ kiểm soát hơn.

4 MARGIN

Tạo khoảng cách bên ngoài border.

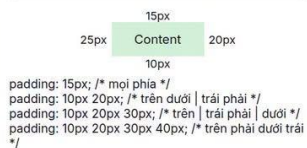
```
div {
  margin: 20px 10px 30px 15px;
}
```



5 PADDING

Tạo khoảng cách bên trong border.

```
div {
  padding: 15px 20px 10px 25px;
}
```



6 BORDER

Định nghĩa đường viền quanh padding.

```
div {
  border: 2px solid #333;
  /* width style color */
}
```

Border Styles



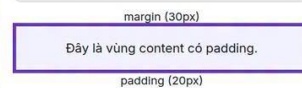
Border Sides

border-top: 2px solid red;
border-right: 2px solid green;
border-bottom: 2px solid blue;
border-left: 2px solid orange;

7 VÍ DỤ ĐẦY ĐỦ

Kết hợp tất cả lại.

```
div {
  width: 250px;
  padding: 20px;
  border: 3px solid #673ab7;
  margin: 30px auto;
  background: #f3f3ff;
  text-align: center;
}
```



MẸO NHANH

- ✓ Dùng margin để tạo khoảng cách giữa các phần tử.
- ✓ Dùng padding để tạo khoảng cách bên trong phần tử.
- ✓ Dùng border-box để layout dễ hơn.
- ✓ Margin có thể collapse (xem trang tiếp theo!). ♥

THƯỜNG DÙNG

```
box-sizing: border-box;
margin: 0 auto; /* căn giữa theo chiều ngang */
padding: 0; /* xóa padding */
border: none; /* xóa border */
```

GHI NHỚ

Content là cái bạn thấy.
Padding là cái bạn cảm nhận.
Margin là cái tạo khoảng cách. 😊

CSS BACKGROUND & BORDER

💡 Dùng background để trang trí phần tử và border để định hình cấu trúc, tạo sự phân tách. ❤️

1 BACKGROUND COLOR

Đặt màu nền cho một phần tử.

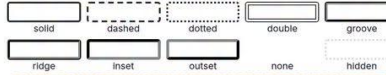
```
div {
  background-color: #f7a7b8;
}
```

VÍ DỤ

Tôi có background color!

5 BORDER STYLE

Định kiểu của border.



Kiểu phổ biến: solid | dashed | dotted | double | groove | ridge | inset | outset | none | hidden

2 BACKGROUND IMAGE

Đặt một hoặc nhiều ảnh nền.

```
div {
  background-image: url('bg.jpg');
}
```

VÍ DỤ



Background Image

Thuộc tính hữu ích:

- background-repeat: repeat | no-repeat | repeat-x | repeat-y
- background-position: top | center | bottom | left | right | x y
- background-size: auto | cover | contain | 100px 100px
- background-attachment: scroll | fixed | local

6 BORDER WIDTH

Đặt độ dày của border.

```
div {
  border-width: 5px;
}
```

Có thể đặt riêng từng cạnh: 1px 2px 3px 4px (trên phải dưới trái)

3 BACKGROUND SHORTHAND

Đặt mọi thuộc tính background trong một dòng.

```
div {
  background: color image repeat position / size
  attachment;
}
```

VÍ DỤ

```
div { background: #f5f5f5 url('bg.jpg') no-repeat center /
  cover fixed; }
```

7 BORDER COLOR

Đặt màu cho border.

```
div {
  border-color: #e63946;
}
```

4 BACKGROUND GRADIENTS

Tạo hiệu ứng chuyển màu mượt mà.

LINEAR GRADIENT

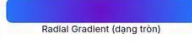
```
background: linear-
gradient(
  to right, #ff7e5f,
  #feb47b
);
```



Linear Gradient (trái sang phải)

RADIAL GRADIENT

```
background: radial-
gradient(
  circle, #6a1b9a, #2575fc
);
```



Radial Gradient (dạng tròn)

8 BORDER RADIUS

Bỏ tròn các góc của border.

```
div {
  border-radius: 15px;
}
```

Đặt riêng từng góc: top-right | bottom-left | top-left | bottom-right
Hoặc: 10px 20px 30px 40px (trên phải dưới trái)

9 BORDER SHORTHAND

Đặt mọi thuộc tính border trong một dòng.

```
div {
  border: 2px dashed #2a9d8f;
}
```

Tôi có border xanh nét đứt!

Cú pháp: border: width style color;

MẸO NHANH

- ✅ Dùng background-size: cover; cho ảnh full màn hình.
- ✅ Dùng no-repeat để tránh lặp ảnh.
- ✅ Kết hợp nhiều background bằng dấu phẩy.

VÍ DỤ: THIẾT KẾ CARD

```
.card {
  background: url('nature.jpg');
  center/cover no-repeat;
  border: 2px solid #fff;
  border-radius: 12px;
  padding: 20px;
  color: #fff;
  box-shadow: 0 4px 15px rgba(0,0,0,0.2);
}
```

Vẻ Đẹp Thiên Nhiên
Khám phá thế giới.
[Xem thêm](#)

GHI NHỚ

- ★ Background tạo sức hút thị giác.
- ★ Border tạo sự phân tách và cấu trúc.
- ★ Dùng border tinh tế cho thiết kế gọn gàng. 😊

CSS DISPLAY, VISIBILITY & OVERFLOW

💡 Các thuộc tính này kiểm soát cách phần tử được hiển thị, người dùng nhìn thấy chúng ra sao, và nội dung tràn được xử lý thế nào. ❤️

1 THUỘC TÍNH DISPLAY

Định nghĩa cách phần tử hiển thị trong tài liệu.

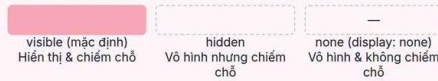
GIÁ TRỊ	MÔ TẢ	VÍ DỤ
block	Chiếm toàn bộ width. Bắt đầu dòng mới.	<code>div { display: block; }</code>
inline	Chỉ chiếm width cần thiết. KHÔNG bắt đầu dòng mới.	<code>span { display: inline; }</code>
inline-block	Giống inline nhưng cho phép width và height.	<code>a { display: inline-block; }</code>
none	Ẩn hoàn toàn phần tử. Xóa khỏi layout.	<code>.hide { display: none; }</code>
flex	Kích hoạt flex container cho các con.	<code>.container { display: flex; }</code>
grid	Kích hoạt grid container cho các con.	<code>.container { display: grid; }</code>

2 THUỘC TÍNH VISIBILITY

Kiểm soát việc phần tử có hiển thị hay không, nhưng vẫn giữ chỗ trong layout.

GIÁ TRỊ	MÔ TẢ	VÍ DỤ
visible	Phần tử hiển thị (mặc định).	<code>.box { visibility: visible; }</code>
hidden	Phần tử vô hình, nhưng vẫn chiếm chỗ.	<code>.box { visibility: hidden; }</code>

VISIBLE vs HIDDEN vs NONE



3 THUỘC TÍNH OVERFLOW

Kiểm soát điều gì xảy ra khi nội dung tràn khỏi hộp của phần tử.

GIÁ TRỊ	MÔ TẢ
visible	Nội dung không bị cắt (mặc định).
hidden	Nội dung bị cắt. Không có scrollbar.
scroll	Nội dung bị cắt, scrollbar luôn hiện.
auto	Scrollbar chỉ hiện khi cần thiết.

4 VÍ DỤ OVERFLOW

visible Đây là nội dung dài tràn ra khỏi hộp.	hidden Đây là nội dung dài tràn ra khỏi hộp.	scroll Đây là nội dung dài tràn ra khỏi hộp.	auto Đây là nội dung dài tràn ra khỏi hộp.
--	---	---	---

```
.box {
  width: 200px;
  height: 100px;
  border: 2px solid #3b82f6;
  overflow: auto;
}
```

Dùng overflow để tạo vùng cuộn và kiểm soát nội dung gọn gàng. 😊

5 KẾT HỢP THƯỜNG DÙNG

display: none

Phần tử bị loại khỏi layout.

Không chiếm chỗ

visibility: hidden

Phần tử ẩn nhưng vẫn chiếm chỗ.

Chiếm chỗ (vô hình)

overflow: hidden

Nội dung thừa bị cắt. Không scrollbar.

Đây là nội dung dài sẽ bị cắt...

overflow: auto

Scrollbar chỉ hiện khi cần.

Đây là nội dung dài có thể cần...

display: inline-block

Hành vi inline kèm tính năng block.

Item 1 Item 2

MẸO NHANH

- ✅ Dùng display để kiểm soát hành vi layout.
- ✅ Dùng visibility khi muốn ẩn nhưng vẫn giữ chỗ.
- ✅ Dùng overflow để quản lý nội dung dư bên trong một hộp. ❤️

VÍ DỤ THỰC TẾ

```
.card {
  width: 250px;
  height: 120px;
  overflow: auto;
  border: 2px solid #8b5cf6;
  padding: 10px;
  border-radius: 8px;
}
```

→

Tiêu đề Card

Đây là nội dung dài bên trong card. Có thể tràn nếu quá nhiều. Dùng thuộc tính overflow để xử lý gọn gàng.

GHI NHỚ

- ★ display: none xóa phần tử khỏi layout.
- ★ visibility: hidden ẩn phần tử nhưng vẫn giữ chỗ.
- ★ overflow giúp xử lý nội dung quá lớn. 😊

CSS POSITIONING & Z-INDEX

5

💡 Positioning trong CSS cho phép bạn kiểm soát vị trí đặt phần tử. z-index kiểm soát thứ tự xếp chồng của các phần tử chồng lên nhau. ❤️

1 CÁC KIỂU POSITIONING

GIÁ TRỊ	MÔ TẢ	ĐỊNH VỊ THEO
static	Giá trị mặc định. Phần tử theo dòng chảy tài liệu bình thường.	(Không áp dụng offset)
relative	Định vị tương đối theo vị trí bình thường của nó.	Vị trí bình thường của nó
absolute	Bị loại khỏi dòng chảy bình thường. Định vị theo tổ tiên gần nhất.	Tổ tiên gần nhất (có position khác static)
fixed	Bị loại khỏi dòng chảy bình thường. Định vị theo viewport.	Viewport (cửa sổ trình duyệt)
sticky	Hành xử như relative đến khi đạt điểm cuộn, rồi dính như fixed.	Tổ tiên có thể cuộn gần nhất (hoặc viewport)

2 POSITIONING HOẠT ĐỘNG THE NÀO

Top, right, bottom, left di chuyển phần tử từ vị trí của nó.

static (mặc định) relative absolute fixed sticky



💡 Mẹo: Để absolute và sticky hoạt động đúng, hãy đảm bảo phần tử cha có position: relative (hoặc khác static).

3 VÍ DỤ POSITIONING



4 GIẢI THÍCH Z-INDEX

z-index hoạt động trên các phần tử đã định vị (relative, absolute, fixed, sticky). z-index cao hơn sẽ nằm trên.

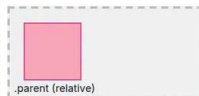


```
.box1 { position: relative; z-index: 1; }
.box2 { position: relative; z-index: 2; }
.box3 { position: relative; z-index: 3; }
```

Nếu z-index bằng nhau, phần tử nào xuất hiện sau trong HTML sẽ nằm trên.

5 VÍ DỤ THỰC TẾ

```
.parent {
  position: relative;
  width: 300px;
  height: 200px;
}
.child {
  position: absolute;
  top: 20px;
  left: 20px;
  width: 100px;
  height: 100px;
}
```



💡 .child được định vị cách top và left 20px so với .parent.

6 TRƯỜNG HỢP DÙNG PHỔ BIẾN

Navigation Bars
Dùng sticky hoặc fixed để giữ menu luôn hiện khi cuộn.

Modals / Popups
Dùng fixed với z-index cao để phủ nội dung.

Tooltips
Dùng absolute so với phần tử cha để đặt chính xác.

Button Back to Top
Dùng fixed để giữ nút luôn hiện ở góc màn hình.

MẸO NHANH

- ✅ Luôn đặt position: relative cho phần tử cha.
- ✅ Chỉ dùng z-index cho phần tử đã định vị.
- ✅ Tránh giá trị z-index quá cao khi không cần thiết. ❤️

STACKING CONTEXT (QUAN TRỌNG!)

Một số thuộc tính tạo ra stacking context mới:

- position (relative/absolute/fixed/sticky) kèm z-index
- opacity nhỏ hơn 1
- transform, filter, perspective
- mix-blend-mode
- isolation: isolate

GHI NHỚ

- ★ Position kiểm soát vị trí đặt.
- ★ z-index kiểm soát thứ tự xếp chồng.
- ★ Hiểu mối quan hệ cha-con.
- ★ Luyện tập để thành thạo! 😊

CSS FLEXBOX

💡 Flexbox là phương pháp layout một chiều để sắp xếp các phần tử theo hàng hoặc cột. Hoàn hảo để xây dựng component. ❤️

1 FLEX CONTAINER

Biến một container thành flex container.

```
.container {
  display: flex;
}
```

VÍ DỤ (row)

1 2 3

FLEX DIRECTION

Đặt hướng của các flex item.
row (mặc định) →
row-reverse →
column ↓
column-reverse ↑

2 JUSTIFY-CONTENT

Căn chỉnh item theo trục chính (ngang mặc định).

flex-start center space-between space-around space-evenly

```
.container {
  display: flex;
  justify-content: center; /*
  thử giá trị khác */
}
```

Giá trị
flex-start (mặc định)
center
flex-end
space-between
space-around
space-evenly

3 ALIGN-ITEMS

Căn chỉnh item theo trục ngang (dọc mặc định).

stretch (mặc định) flex-start center flex-end baseline

```
.container {
  display: flex;
  align-items: center; /* thử
  giá trị khác */
}
```

Giá trị
stretch (mặc định)
flex-start
center
flex-end
baseline

4 FLEX-WRAP

Kiểm soát việc item có xuống dòng hay không.

nowrap (mặc định) wrap wrap-reverse

```
.container {
  display: flex;
  flex-wrap: wrap; /* nowrap | wrap | wrap-reverse */
}
```

💡 Mẹo: Dùng wrap cho layout responsive và component linh hoạt.

5 ALIGN-CONTENT

Căn nhiều dòng theo trục ngang. (Chỉ hoạt động khi flex-wrap: wrap;)

stretch flex-start center space-between space-around

```
.container {
  display: flex;
  flex-wrap: wrap;
  align-content: space-between; /* thử giá trị khác */
}
```

6 FLEX ITEMS (CON)

FLEX-GROW

Item sẽ giãn ra bao nhiêu để lấp đầy.

```
.item { flex-grow: 1; /* 0 mặc
  định */ }
```

FLEX-SHRINK

Item sẽ co lại bao nhiêu khi cần.

```
.item { flex-shrink: 1; /* 1
  mặc định */ }
```

FLEX-BASIS

Kích thước ban đầu trước khi chia khoảng trống.

```
.item { flex-basis: 100px; }
```

7 ALIGN-SELF

Căn chỉnh riêng một item theo trục ngang. (Ghi đè align-items cho item đó.)

Giá trị: auto | flex-start | center | flex-end | stretch | baseline

```
.item {
  align-self: center; /* ghi đè align-
  items */
}
```

8 ORDER

Kiểm soát thứ tự hiển thị của item. (Mặc định là 0)

```
.item {
  order: 2; /* -1 đến trước, 1 tiếp
  theo, 2 cuối cùng */
}
```

Thứ tự HTML: A B C ... Thứ tự hiển thị: B A C

9 VÍ DỤ NHANH (NAVBAR)

Navbar responsive đơn giản dùng Flexbox.

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

Logo

Home About Dịch vụ Liên hệ

MẸO NHANH

- ✅ Dùng Flexbox cho layout một chiều.
- ✅ Flexbox hoàn hảo để căn giữa item.
- ✅ Kết hợp thuộc tính để có layout mạnh mẽ. ❤️

CÁC THUỘC TÍNH FLEX CONTAINER PHỔ BIẾN

```
display: flex
flex-direction: row | column
flex-wrap: nowrap | wrap | wrap-reverse
justify-content: center | space-between | ...
align-items: center | flex-start | ...
align-content: center | space-between | ...
```

GHI NHỚ

- ★ Flexbox = layout 1 chiều (hàng hoặc cột).
- ★ Cha là flex container.
- ★ Con là flex item.
- ★ Kiểm soát canh chỉnh, khoảng cách & thứ tự dễ dàng! 😊

CSS CSS GRID

💡 CSS Grid Layout là hệ thống layout hai chiều cho web. Nó cho phép bạn thiết kế layout phức tạp với hàng và cột. ❤️

1 GRID CONTAINER

Đặt display: grid trên phần tử cha.

```
.container {  
  display: grid;  
}
```

Con trực tiếp của grid container trở thành grid item.



Thuộc tính thường dùng: display: grid | grid-template-columns | grid-template-rows | gap | justify-items | align-items | place-items

2 GRID TEMPLATE COLUMNS

Định nghĩa số lượng và kích thước của các cột.

Cột bằng nhau Cột định & Cố định Đơn vị hỗn hợp

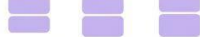
```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr; /* thử giá trị khác */  
  gap: 16px;  
}
```

Giá trị có thể dùng: px | % | fr | auto | minmax()

3 GRID TEMPLATE ROWS

Định nghĩa số lượng và kích thước của các hàng.

Hàng cố định Hàng hỗn hợp Hàng tự động



```
.container {  
  display: grid;  
  grid-template-rows: 100px auto 1fr; /* thử giá trị khác */  
  gap: 16px;  
}
```

4 GAP (KHOẢNG CÁCH)

Tạo khoảng cách giữa các hàng và cột.

gap (hàng & cột) row-gap (dọc) column-gap (ngang)



```
.container { display: grid; gap: 20px; /* hoặc */ row-gap:  
20px; column-gap: 30px; }
```

💡 gap là shorthand của row-gap và column-gap.

5 JUSTIFY-ITEMS (CĂN NGANG)

Căn item theo chiều ngang bên trong ô grid.

start (mặc định) center end stretch

```
.container { display: grid; justify-items: center; /* thử giá  
trị khác */ }
```

6 ALIGN-ITEMS (CĂN DỌC)

Căn item theo chiều dọc bên trong ô grid.

start (mặc định) center end stretch

```
.container { display: grid; align-items: center; /* thử giá  
trị khác */ }
```

7 PLACE-ITEMS (SHORTHAND)

Shorthand cho align-items và justify-items.

```
.container { display: grid; place-items: center; /* căn cả 2  
trục */ }
```

Giá trị: start | center | end | stretch

8 SPAN (KÍCH THƯỚC ITEM)

Cho item chiếm nhiều cột hoặc hàng.

Column Span Row Span

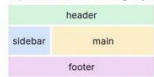
1 (span) 1

3 4 5 1 2

```
.item1 { grid-column: span 2; }  
.item2 { grid-row: span 2; }
```

9 AREAS (VÙNG GRID CÓ TÊN)

Đặt tên cho các vùng layout để cấu trúc rõ ràng hơn.



```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  gap: 12px;  
}  
.header { grid-area: header; }
```

10 AUTO-FLOW (SẮP XẾP)

Kiểm soát cách các item tự động được thêm vào.

row (mặc định) column

1 2 4 1 5 4 6 7 8

5 6 8

```
.container { display: grid; grid-auto-flow: row; /* hoặc column  
*/ gap: 12px; }
```

MẸO NHANH

- ✔ Grid hoàn hảo cho layout 2D phức tạp.
- ✔ Dùng fr cho layout linh hoạt, responsive.
- ✔ Dùng minmax() cho track thông minh, linh hoạt.
- ✔ Kết hợp grid areas + span để có sức mạnh! ❤️

ĐƠN VỊ THƯỜNG DÙNG TRONG GRID

px - Kích thước cố định
% - Phần trăm theo container
fr - Phần không trống còn lại
auto - Kích thước theo nội dung
minmax(min, max) - Khoảng kích thước linh hoạt

GHI NHỚ

- ★ Grid = Hàng + Cột.
- ★ Con trực tiếp là grid item.
- ★ Bạn kiểm soát cả 2 chiều.
- ★ Luyện tập và xây dựng layout tuyệt vời!

CSS PSEUDO-CLASSES & PSEUDO-ELEMENTS



💡 Pseudo-class style phần tử dựa trên trạng thái. Pseudo-element style một phần cụ thể của phần tử. ❤️

1 PSEUDO-CLASSES (:)

Nhắm vào phần tử ở một trạng thái cụ thể.

SELECTOR	MÔ TẢ
:hover	Khi người dùng di chuột qua phần tử.
:active	Khi phần tử đang được click.
:focus	Khi phần tử có focus bàn phím.
:visited / :link	Style khác nhau cho link đã/chưa xem.
:first-child	Con đầu tiên của phần tử cha.
:last-child	Con cuối cùng của phần tử cha.
:nth-child(n)	Chọn con thứ n.
:not(selector)	Chọn phần tử không khớp.

2 VÍ DỤ PSEUDO-CLASSES

Hover Tôi
 Đôi khi di chuột qua.

Click Tôi
 Đôi khi bạn click.

Focus tại đây
 Đôi khi có focus (Tab).

Link chưa xem
 Link đã xem
 Style khác cho đã/chưa xem.

- Item 1 (đậm)
- Item 2
- Item 3

 Style con đầu tiên.

- Item 1
- Item 2
- Item 3 (đậm)

 Style con cuối cùng.

- Item 1
- Item 2 (đậm)
- Item 3

 Style con thứ 2.

Active
 Not Active
 Chọn phần tử không có .active.

💡 Nhiều pseudo-class */

```
a:hover, a:focus { color: #e91e63; text-decoration: underline; }
```

3 PSEUDO-ELEMENTS (::)

Style một phần cụ thể của phần tử.

SELECTOR	MÔ TẢ
::before	Chèn nội dung trước phần tử.
::after	Chèn nội dung sau phần tử.
::first-letter	Style chữ cái đầu của phần tử.
::first-line	Style dòng đầu của phần tử.
::selection	Style phần text được bôi chọn.

VÍ DỤ TRỰC QUAN

```
::before
★ Heading

::after
Heading [M]

::first-letter
Đây là một đoạn văn.

::selection
Chọn bất kỳ text nào
```

4 KẾT HỢP PSEUDO-CLASSES

Nối chuỗi các pseudo-class để style cụ thể hơn.

```
a:hover {
  color: #e91e63;
  text-decoration: underline;
}
a:focus {
  outline: 2px dashed #e91e63;
}
```

Hover hoặc focus mình!
Thử di chuột hoặc tab bằng bàn phím.

5 KẾT HỢP PSEUDO-ELEMENTS

Dùng cả ::before và ::after cùng nhau.

```
.quote::before {
  content: "\201C"; /* dấu ngoặc mở */
}
.quote::after {
  content: "\201D"; /* dấu ngoặc đóng */
}
```

Đây là câu trích dẫn truyền cảm hứng dùng pseudo-elements!

6 TRƯỜNG HỢP DÙNG PHỔ BIẾN

Icon trước Link Tooltips Badges / Nhãn Drop Caps

🏠 Home Hover mình Inbox 5 L orem ipsum dolor sit amet.

MẸO NHANH

- ✅ Dùng :hover để phản hồi tương tác.
- ✅ Dùng :focus để hỗ trợ khả năng tiếp cận.
- ✅ Dùng ::before và ::after cho icon/nội dung trang trí.
- ✅ ::selection cải thiện trải nghiệm người dùng.

THỨ TỰ PSEUDO-CLASS (LVHA)

Dùng thứ tự này cho link:
/* Đúng */
a:link → a:visited → a:hover → a:active

GHI NHỚ

- ★ Pseudo-class = trạng thái phần tử.
- ★ Pseudo-element = một phần của phần tử.
- ★ Giúp viết CSS mạnh mẽ & gọn gàng.
- ★ Cần thiết cho thiết kế web hiện đại! ❤️

CSS TRANSITIONS, TRANSFORMS & ANIMATIONS



💡 Các thuộc tính này thêm chuyển động và hiệu ứng cho website, giúp nó sinh động và hấp dẫn hơn. ❤️

1 TRANSITIONS

Transition cho phép thay đổi giá trị thuộc tính mượt mà trong một khoảng thời gian.

```
.box {
  width: 100px;
  background: #6c5ce7;
  transition: property duration timing-function delay;
}
```

Trạng thái Bình thường



Khi Hover
(chuyển mượt) →

Trạng thái Hover



HÀM TIMING
ease / linear / ease-in / ease-out /
ease-in-out

2 TRANSFORMS

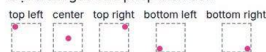
Transform cho phép xoay, phóng to/thu nhỏ, nghiêng hoặc di chuyển phần tử.

HÀM	VÍ DỤ	KẾT QUẢ
translate()	translate(50px, 20px)	
rotate()	rotate(45deg)	
scale()	scale(1.5)	
skewX()	skewX(20deg)	
skewY()	skewY(20deg)	

💡 Bạn có thể kết hợp nhiều transform: transform: translate(20px, 10px) rotate(30deg) scale(1.2);

3 TRANSFORM ORIGIN

Đặt điểm gốc cho phép biến đổi.



```
.box { transform-origin: center; /* mặc định là center */ }
```

Từ khóa: top, left, center, bottom, right
Phần trăm: 0% 0%, 50% 50%, 100% 100%
Độ dài: 20px 30px

4 ANIMATIONS

Animation cho phép tạo hiệu ứng dựa trên keyframe.

```
@keyframes move {
  0% { left: 0; }
  50% { left: 100px; }
  100% { left: 200px; }
}
```

```
.box {
  animation-name: move;
  animation-duration: 2s;
  animation-iteration-count:
  infinite;
}
```

0% → 50% → 100%

Thuộc tính: animation-name, duration, timing-function, delay, iteration-count, direction, fill-mode, play-state

5 TRANSITION vs ANIMATION

TRANSITION

- Kích hoạt bởi thay đổi trạng thái (hover, focus...).
- Đi từ trạng thái này sang trạng thái khác.
- Không dùng keyframe.
- Ví dụ: hiệu ứng hover nút.

ANIMATION

- Chạy tự động (hoặc theo trigger).
- Có thể có nhiều keyframe.
- Kiểm soát chuỗi tốt hơn.
- Ví dụ: spinner loading.

VS

6 VÍ DỤ TRANSITION THỰC TẾ

Nút với hiệu ứng hover mượt mà.

BUTTON BUTTON BUTTON
Bình thường Hover (Scale + Màu) Hover (Đỏ bóng)

```
.btn { padding: 10px 20px; background: #6c5ce7; transition: all 0.3s ease; }
.btn:hover { background: #5f3dc4; transform: scale(1.05); }
```

7 VÍ DỤ ANIMATION THỰC TẾ

Hiệu ứng animation chấm này loading.



```
@keyframes bounce { 0%, 100% { transform: translateY(0); } 50% { transform: translateY(-15px); } }
.dot { animation: bounce 1s ease-in-out infinite; }
```

MẸO NHANH

- ✓ Dùng transition cho thay đổi trạng thái đơn giản.
- ✓ Dùng animation cho chuỗi phức tạp.
- ✓ Luôn test trên nhiều trình duyệt. ❤️

GHI NHỚ

- ★ Animation nhỏ = ấn tượng lớn!
- ★ Đừng lạm dụng animation.
- ★ Hiệu năng rất quan trọng. Dùng transform & opacity cho mượt hơn.

CSS RESPONSIVE CSS & MEDIA QUERIES



💡 Responsive design giúp website của bạn đẹp trên mọi thiết bị. Media query giúp bạn áp dụng style khác nhau tùy điều kiện thiết bị. ❤️

1 TẠI SAO CẦN RESPONSIVE DESIGN?

📱 Cải thiện trải nghiệm người dùng trên mọi thiết bị 📈 Tăng thứ hạng SEO 🔄 Tiếp cận đối tượng rộng hơn ⚡ Giảm nhu cầu làm nhiều phiên bản

2 CÚ PHÁP MEDIA QUERY

```
@media media-type and (condition)
{
  /* CSS rules */
}
```

THUỘC TÍNH PHỔ BIẾN

- width
- min-width
- max-width
- height
- orientation
- resolution

media-type → all | screen | print | speech
condition → dựa trên width, height, orientation...

3 BREAKPOINT PHỔ BIẾN

THIẾT BỊ	MIN-WIDTH	MAX-WIDTH	VÍ DỤ
Extra Small (Mobile)	0px	575.98px	Điện thoại (dọc)
Small (Mobile Landscape)	576px	767.98px	Điện thoại lớn, tablet nhỏ
Medium (Tablet)	768px	991.98px	Tablet, iPad
Large (Laptop)	992px	1199.98px	Laptop, desktop nhỏ
Extra Large (Desktop)	1200px	trở lên	Desktop, màn hình lớn

💡 Đây chỉ là gợi ý. Hãy chọn breakpoint phù hợp với thiết kế của bạn.

4 VÍ DỤ MEDIA QUERY

MỤC ĐÍCH	MEDIA QUERY	ÁP DỤNG CHO
Mobile First	@media (min-width: 0px)	Mọi thiết bị
Small Devices	@media (min-width: 576px)	576px trở lên
Tablets	@media (min-width: 768px)	768px trở lên
Laptops	@media (min-width: 992px)	992px trở lên
Desktops	@media (min-width: 1200px)	1200px trở lên

💡 Dùng min-width là cách tiếp cận mobile-first (khuyến nghị).

5 CÁCH TIẾP CẬN MOBILE-FIRST (KHUYẾN NGHỊ)

Bước 1: Style cho mobile (mặc định) → Bước 2: Thêm media query cho màn hình lớn hơn

```
/* Style cơ bản (cho mobile) */
.container { padding: 15px; font-size: 16px; }
/* Cho tablet trở lên */
@media (min-width: 768px) {
  .container { padding: 30px; font-size: 18px; }
}
/* Cho laptop trở lên */
@media (min-width: 992px) {
  .container { padding: 40px; font-size: 20px; }
}
```

TIPS

- ✓ Bắt đầu với style mobile.
- ✓ Nâng cấp dần cho màn hình lớn hơn.
- ✓ Tránh dùng max-width nếu có thể.

6 VÍ DỤ LAYOUT RESPONSIVE

MOBILE (< 576px)	TABLET (> 768px)	DESKTOP (> 992px)
Header	Header	Header
Menu	Menu	Menu
Main Content	Main Sidebar	Main Sidebar
Sidebar	Footer	Footer
Footer		

```
/* Ví dụ: thay đổi layout ở màn hình lớn */
.container { display: block; }
@media (min-width: 768px) {
  .container { display: grid; grid-template-columns: 2fr 1fr; gap: 20px; }
}
```

7 CÁC TÍNH NĂNG MEDIA HỮU ÍCH KHÁC

TÍNH NĂNG	VÍ DỤ	MÔ TẢ
max-width	(max-width: 767.98px)	Áp dụng đến width cho trước
orientation	(orientation: landscape)	Chế độ ngang hoặc dọc
resolution	(min-resolution: 2dppx)	Màn hình mật độ cao / retina
prefers-color-scheme	(prefers-color-scheme: dark)	Người dùng thích chế độ tối

✓ BEST PRACTICES

- ✓ Dùng đơn vị tương đối (% , em, rem, vw, vh).
- ✓ Làm ảnh và media responsive (max-width: 100%).
- ✓ Test trên thiết bị thật và nhiều kích thước.
- ✓ Giữ breakpoint nhất quán.
- ✓ Thiết kế theo nội dung, không theo thiết bị.

🔥 GHI NHỚ

- ★ Responsive design = trải nghiệm người dùng tốt hơn.
- ★ Media query = style khác nhau cho điều kiện khác nhau.
- ★ Mobile-first = viết ít, làm được nhiều.
- ★ Test, điều chỉnh & tối ưu! 😊

CSS

CSS NÂNG CAO - CASCADE, SPECIFICITY & VARIABLES



💡 Các khái niệm này kiểm soát style nào được áp dụng, một selector mạnh đến đâu, và cách tái sử dụng giá trị dễ dàng. ❤️

1 CASCADE - THỨ TỰ CỦA CÁC STYLE

Khi nhiều rule cùng nhắm vào một phần tử, style cuối cùng được quyết định bởi cascade.

THỨ TỰ CASCADE (Thấp -- Cao)

1. Style mặc định của trình duyệt
2. Style của người dùng
3. Style của tác giả (external, internal, inline)
4. Khai báo !important
5. Inline style

```
/* Màu nào sẽ được áp dụng? */
p { color: blue; }
.text { color: green; }
#main p { color: red; }

<p id="main" class="text"
  style="color: purple;">
  Hello!
</p>
```

Đáp án: purple (inline style thắng)

2 SPECIFICITY - TRÌNH DUYỆT TÍNH ĐỘ MẠNH RA SAO

Mỗi selector có điểm specificity theo định dạng: (A, B, C, D)

A	B	C	D
Inline styles (1,0,0,0) style="..."	IDs (0,1,0,0) #id	Class, attribute, pseudo-class (0,0,1,0) .class, [type], :hover	Element, pseudo- element (0,0,0,1) div, p, ::before

SELECTOR	SPECIFICITY	GIẢI THÍCH	TRỌNG SỐ
*	(0,0,0,0)	Universal selector	0
p	(0,0,0,1)	Element	1
.text	(0,0,1,0)	Class	10
p.text	(0,0,1,1)	Element + Class	11
#main	(0,1,0,0)	ID	100
#main .text	(0,1,1,0)	ID + Class	110
p#main.text:hover	(0,1,2,1)	ID + Class + Pseudo-class + Element	121
style="..."	(1,0,0,0)	Inline style	1000

💡 Nếu specificity bằng nhau, rule nào xuất hiện sau trong CSS sẽ thắng.

3 !IMPORTANT - DÙNG THẬN TRỌNG

Ghi đè cascade và specificity thông thường (trừ inline styles).

```
.btn {
  color: blue !important;
}
```

Màu nào?

```
.btn { color: blue !important; }
#main .btn { color: red; }
<button class="btn">Click</button>
```

Đáp án: blue (vì !important)

BEST PRACTICES

- ✓ Chỉ dùng khi cần thiết.
- ✓ Tránh lạm dụng.
- ✓ Nó khiến CSS khó bảo trì hơn.

⚠️ CẢNH BÁO

Chỉ inline style mới có thể ghi đè !important.
<button style="color: green;">Click</button>

4 CSS VARIABLES (CUSTOM PROPERTIES)

Lưu giá trị trong biến tài sử dụng. Bắt đầu bằng --

CÚ PHÁP

```
/* 1. Khai báo */
:root {
  --primary:
    #6366f1;
  --text: #333333;
  --radius: 8px;
}
```

VÍ DỤ

```
/* 2. Sử dụng */
.btn {
  background: var(--
    primary);
  color: white;
  border-radius:
    var(--radius);
}
```

TẠI SAO DÙNG VARIABLES?

- 🔒 Để bảo trì
- 🔄 Thiết kế nhất quán
- 🔄 Chuyển theme
- 🔄 It lập lại

GIÁ TRỊ DỰ PHÒNG

color: var(--text, #000); /* Dùng #000 nếu --text không được định nghĩa */

5 VAR() KẾT HỢP TÍNH TOÁN & HÀM

VỚI CALC()

```
:root { --spacing:
  10px; }
.box { padding:
  calc(var(--spacing)
    * 2); /* 20px */ }
```

VỚI RGB / OPACITY

```
:root { --color: 99,
  182, 241; }
.box { background:
  rgba(var(--color),
    0.2); }
```

VỚI HSL

```
:root { --hue: 220; }
.box { background:
  hsl(var(--hue), 80%,
    60%); }
```

💡 Dùng variables cho spacing, màu sắc, font, border-radius, shadow, v.v.

6 CASCADE ÁP DỤNG THỰC TẾ (VÍ DỤ)

Màu nào sẽ được áp dụng cho text?

```
p { color: blue; } /* (0,0,0,1) */
.text { color: green; } /* (0,0,1,0) */
#main p { color: red; } /* (0,1,0,1) */
p.text { color: orange; } /* (0,0,1,1) */

<p id="main" class="text" style="color:
  purple;">Hello CSS!</p>
```

PHÂN TÍCH

1. Inline style (purple) -- cao nhất
2. #main p (red) -- tiếp theo
3. p.text (orange) -- tiếp theo
4. text (green) -- tiếp theo
5. p (blue) -- thấp nhất

KẾT QUẢ: PURPLE 🏆

7 VÍ DỤ THỰC TẾ: THEME DÙNG VARIABLES

LIGHT THEME (MẶC ĐỊNH)

```
:root {
  --bg: #ffffff;
  --text: #111827;
  --primary: #6366f1;
}
```

DARK THEME

```
@media (prefers-
  color-scheme: dark) {
  :root {
    --bg: #f1f7fa;
    --text: #5e7eb8;
  }
}
```

SỬ DỤNG

```
body {
  background: var(--
    bg);
  color: var(--text);
}
.btn {
  background: var(--
    primary);
}
```

TÓM TẮT NHANH

- ✓ Cascade quyết định rule nào được áp dụng.
- ✓ Specificity xác định độ mạnh của selector.
- ✓ !important ghi đè (dùng thận trọng).
- ✓ Variables giúp CSS tái sử dụng & mở rộng. ❤️

📌 GHI NHỚ

- ★ Inline > !important > IDs > Classes > Elements
- ★ Giữ specificity ở mức thấp.
- ★ Dùng variables cho code sạch & dễ bảo trì.
- ★ Viết một lần, dùng ở mọi nơi! ❤️

CSS TỔNG HỢP KIẾN THỨC

Các thẻ, thuộc tính, ký tự đặc biệt thường dùng + kiểu input + cấu trúc trang mini + sơ đồ ôn tập cuối

1 CÁC THẸ HTML PHỔ BIẾN

<div>	Khởi chứa nội dung
	Khởi nội tuyến (inline)
<p>	Đoạn văn
<a>	Liên kết
	Hình ảnh
	Danh sách không thứ tự
	Danh sách có thứ tự
	Mục trong danh sách
<table>	Bảng
<tr>	Hàng của bảng
<th>	Tiêu đề của bảng
<td>	Ô dữ liệu của bảng
<form>	Khởi chứa form
<input>	Điều khiển nhập liệu
<button>	Nút bấm
<header>	Tiêu đề trang hoặc mục
<nav>	Liên kết điều hướng
<main>	Nội dung chính
<section>	Phần trong trang
<article>	Nội dung độc lập
<aside>	Nội dung bên lề (sidebar)
<footer>	Chân trang

2 THUỘC TÍNH PHỔ BIẾN

id	Định danh duy nhất
class	Nhóm các phần tử
style	Style CSS inline
href	Tham chiếu liên kết (cho <a>)
src	Nguồn (cho , <script>)
alt	Text thay thế (cho)
title	Thông tin thêm khi hover
target	Nơi mở liên kết (_blank, _self)
rel	Quan hệ (stylesheet, opener)
type	Kiểu (cho input, button...)
name	Tên cho các control của form
value	Giá trị mặc định
placeholder	Gợi ý text
required	Bắt buộc phải điền
disabled	Vô hiệu hóa phần tử
readonly	Chỉ đọc (không sửa được)
min / max	Giá trị nhỏ nhất / lớn nhất
step	Giá trị bước nhảy
checked	Được chọn sẵn (checkbox/radio)
maxlength	Độ dài tối đa của input
autocomplete	Bật/tắt tự động hoàn thành

3 HTML ENTITIES (VÍ DỤ)

<	<	Nhỏ hơn	©	©	Bản quyền
>	>	Lớn hơn	®	®	Đã đăng ký
&	&	Dấu và	€	€	Euro
"	"	Ngược kép	™	™	Nhãn hiệu
'	'	Ngược đơn	×	x	Nhân
 		Khoảng trắng cứng	÷	÷	Chia

4 KÝ TỰ ĐẶC BIỆT (DÙNG TRONG NỘI DUNG CSS)

~ Mũi tên phải	\2192	✓ Dấu tích	\2713
~ Mũi tên trái	\2190	✗ Dấu chéo	\2717
~ Mũi tên lên	\2191	★ Ngôi sao	\2605
~ Mũi tên xuống	\2193	◉ Ngôi sao (viên)	\2606
~ Phong bì	\2709	♥ Trái tim	\2665
☎ Điện thoại	\260E	◆ Diamond	\2666

5 CÁC KIỂU INPUT (CHO <input>)

text	Nhập text	datetime-local	Ngày & giờ
password	Mật khẩu	month	Tháng
email	Email	week	Tuần
number	Số	color	Chọn màu
tel	Số điện thoại	range	Thanh trượt
url	URL	checkbox	Checkbox
search	Ô tìm kiếm	radio	Radio button
date	Ngày tháng	file	Tải file lên
time	Thời gian	hidden	Ẩn

6 CẤU TRÚC TRANG MINI (HTML5)

<header>	<!DOCTYPE html>
<nav>	<html lang="en">
<main>	<head>
<section>	<meta charset="UTF-8">
<article>	<meta name="viewport" content="width=device-width, initial-scale=1.0">
<footer>	<title>Trang Của Tôi</title>
	<link rel="stylesheet" href="style.css">
	</head>
	<body>
	<header><header></header>
	<nav><navigation></nav>
	<main>
	<aside><sidebar></aside>
	<section>
	<article>Nội dung...</article>
	</section>
	</main>
	<footer><footer></footer>
	</body>
	</html>

7 SƠ ĐỒ ÔN TẬP CUỐI (CHEAT MAP)

Selectors Type, Class, ID Attribute, Pseudo Combinators	Box Model Content Padding Border Margin	Layout Display Position Flexbox Grid	Responsive Media Queries Breakpoints Flexible Units Mobile-First
Typography Font Text Line-height Letter-spacing	Visuals Backgrounds Borders Shadows Filters	Effects Transitions Transforms Animations Opacity	Nâng cao Cascade Specificity Variables Functions

CSS

MẸO NHANH

- ✓ Dùng class để tái sử dụng, ID cho phần tử duy nhất.
- ✓ Mobile-first design là best practice.
- ✓ Giữ CSS gọn gàng và có chủ thích rõ ràng.
- ✓ Dùng variables cho màu sắc và giá trị thường dùng. ❤️

ĐƠN VỊ PHỔ BIẾN

px - Pixel cố định
% - Phần trăm
em - Tương đối cha
rem - Tương đối gốc
vh - 1% chiều cao viewport
vw - 1% chiều rộng viewport

vmin - Nhỏ nhất của vw/vh
vmax - Lớn nhất của vw/vh
ch - Độ rộng của "0"
ex - Chiều cao của "x"
pt - Đơn vị in ấn
cm/mm/in - Đơn vị in ấn

GHI NHỚ

- ★ Từng bước nhỏ và đều đặn tạo nên thiết kế tuyệt vời.
- ★ Hiểu sâu những điều cơ bản.
- ★ Luyện tập mỗi ngày.
- ★ Xây dựng dự án và học qua thực hành! 🚀